

JavaScript 데이터 처리 도구 전체 구조

코딩테스트에서 자주 쓰는 JavaScript 자료구조와 메서드의 전체 구조를 한눈에 파악한다.

1. JavaScript 데이터 처리 도구 — 전체 구조

JavaScript Data Processing Tools

- └ Array – Java의 ArrayList + Stream 역할을 동시에 수행
 - └ 데이터 저장/조작: push, pop, splice, slice, indexOf, includes ...
 - └ 정렬: sort(callback) – callback이 Java의 Comparator 역할
 - └ 고차 함수: filter, map, reduce, find, some, every ... – Java Stream
- 역할
- └ Object – Java HashMap과 유사한 Key-Value 저장소 (간단 버전)
 - └ Object.keys / values / entries, obj[key] 접근
- └ Map – Java HashMap에 더 가까움 (순서 보장, 어떤 타입이든 키 가능)
 - └ set, get, has, size, entries
- └ Set – Java HashSet에 대응 (중복 제거)
 - └ add, has, size, delete
- └ Promise / async-await – 비동기 처리 (JS 고유, Java에 직접 대응 없음)
 - └ 코딩테스트에서 비동기 콜백 패턴으로 출제 가능

2. Java와의 핵심 구조적 차이

관점	Java	JavaScript
계층 구조	엄격한 인터페이스 계층 (Collection → List → ArrayList)	Array가 거의 모든 것을 처리, 프로토타입 기반
Stream	컬렉션과 별도의 API (<code>collection.stream()</code>)	Array 메서드 자체가 Stream (체이닝 가능)
타입 시스템	정적 타입 (컴파일 타임 검사)	동적 타입 (런타임에 결정)
정렬	Comparator 인터페이스	sort() 콜백 함수
수집	<code>.collect(Collectors.toList())</code> 필요	filter/map 결과가 바로 배열

3. ES5 → ES6(ES2015) 그리고 그 이후 — 왜 ES6가 중요한가

ES6 이전 (ES5)

- **var**만 존재 — 호이스팅과 함수 스코프 문제
- 콜백 지옥(callback hell) — 비동기 처리의 가독성 최악
- **function** 선언만 가능 — **this** 바인딩 문제

ES6 (2015)에서 추가된 것들

기능	왜 필요했나
let/const	var의 호이스팅/스코프 문제 해결 (블록 스코프)
화살표 함수	this 바인딩 문제 해결 + 간결한 콜백 작성
Map/Set	Object의 한계 (키가 문자열만 가능) 극복
Promise	콜백 지옥 해결
구조분해 할당	객체/배열에서 값을 쉽게 추출
스프레드 연산자	배열/객체 복사, 합치기 간편화
템플릿 리터럴	문자열 보간(interpolation) 지원

ES2017: async/await

- Promise 체이닝의 가독성을 한층 더 개선

비교 1: 콜백 함수 — function vs 화살표 함수

```
// === ES5: function 키워드 ===
const nums1 = [1, 2, 3];
const doubled1 = nums1.map(function(n) {
  return n * 2;
});
console.log(doubled1); // [2, 4, 6]

// === ES6: 화살표 함수 ===
const nums2 = [1, 2, 3];
const doubled2 = nums2.map(n => n * 2);
console.log(doubled2); // [2, 4, 6]
```

비교 2: 반복 — for문 vs forEach/map

```
const fruits = ["apple", "banana", "cherry"];

// === ES5: for문 ===
const upper1 = [];
for (let i = 0; i < fruits.length; i++) {
  upper1.push(fruits[i].toUpperCase());
}
console.log(upper1); // ["APPLE", "BANANA", "CHERRY"]
```

```
// === ES6: map ===
const upper2 = fruits.map(f => f.toUpperCase());
console.log(upper2); // ["APPLE", "BANANA", "CHERRY"]
```

비교 3: 비동기 — 콜백 지옥 vs Promise vs async/await

```
// === 콜백 지옥 ===
// getUser(1, function(user) {
//     getPosts(user.id, function(posts) {
//         getComments(posts[0].id, function(comments) {
//             console.log(comments); // 들여쓰기 지옥
//         });
//     });
// });

// === Promise 체이닝 ===
function getUser(id) {
    return new Promise(resolve => {
        setTimeout(() => resolve({ id, name: "Alice" }), 100);
    });
}

function getPosts(userId) {
    return new Promise(resolve => {
        setTimeout(() => resolve([{ id: 1, title: "Post 1" }]), 100);
    });
}

function getComments(postId) {
    return new Promise(resolve => {
        setTimeout(() => resolve(["comment1", "comment2"]), 100);
    });
}

// Promise 체이닝
getUser(1)
    .then(user => getPosts(user.id))
    .then(posts => getComments(posts[0].id))
    .then(comments => console.log(comments)); // ["comment1", "comment2"]

// === async/await (가장 깔끔) ===
async function fetchData() {
    const user = await getUser(1);
    const posts = await getPosts(user.id);
    const comments = await getComments(posts[0].id);
    console.log(comments); // ["comment1", "comment2"]
}

fetchData();
```

4. 비동기 기초 — 코딩테스트 출제 가능

콜백 함수란?

- 다른 함수에 인자로 전달되어 나중에 호출되는 함수
- JavaScript의 핵심 패턴 — 이벤트 처리, 비동기 처리 모두 콜백 기반

```
// 콜백 함수의 가장 기본적인 형태
function greet(name, callback) {
  console.log("안녕, " + name);
  callback();
}

greet("Alice", () => console.log("인사 끝!"));
// 안녕, Alice
// 인사 끝!
```

Promise 3가지 상태

상태	의미
pending	아직 완료되지 않음 (초기 상태)
fulfilled	성공적으로 완료 → <code>.then()</code> 으로 처리
rejected	실패 → <code>.catch()</code> 로 처리

```
// 성공하는 Promise
const success = new Promise((resolve, reject) => {
  setTimeout(() => resolve("성공!"), 100);
});

success.then(result => console.log(result)); // 성공!

// 실패하는 Promise
const fail = new Promise((resolve, reject) => {
  setTimeout(() => reject("에러 발생!"), 100);
});

fail.catch(error => console.log(error)); // 에러 발생!
```

async/await 기본 패턴

```
// async 함수는 항상 Promise를 반환한다
async function fetchNumber() {
  return 42;
}
```

```

fetchNumber().then(n => console.log(n)); // 42

// await는 Promise가 resolve될 때까지 기다린다
async function process() {
  const result = await new Promise(resolve => {
    setTimeout(() => resolve("완료"), 100);
  });
  console.log(result); // 완료
}

process();

```

Promise.all / Promise.race

```

// Promise.all - 모든 Promise가 완료될 때까지 대기
async function fetchAll() {
  const results = await Promise.all([
    new Promise(resolve => setTimeout(() => resolve("A"), 300)),
    new Promise(resolve => setTimeout(() => resolve("B"), 100)),
    new Promise(resolve => setTimeout(() => resolve("C"), 200)),
  ]);
  console.log(results); // ["A", "B", "C"] - 순서 보장
}

fetchAll();

// Promise.race - 가장 먼저 완료된 결과만
async function fetchRace() {
  const fastest = await Promise.race([
    new Promise(resolve => setTimeout(() => resolve("느림"), 300)),
    new Promise(resolve => setTimeout(() => resolve("빠름"), 100)),
  ]);
  console.log(fastest); // 빠름
}

fetchRace();

```

코딩테스트 출제 패턴 예시

```

// 패턴 1: 실행 순서 맞추기
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// 출력 순서: 1 → 4 → 3 → 2
// 이유: 동기 → 마이크로태스크(Promise) → 매크로태스크(setTimeout)

// 패턴 2: 비동기 반복 처리
async function processItems(items) {

```

```
// 순차 처리 (하나씩)
for (const item of items) {
  const result = await new Promise(resolve =>
    setTimeout(() => resolve(item * 2), 10)
  );
  console.log(result);
}

processItems([1, 2, 3]); // 2, 4, 6 (순서 보장)
```